

SD TCL - Select Lists

Choosing a set of records once and then using it, saving it, and combining it with another.

A select list is a list of record ids that SD is holding for you. You build one with `select`, and **the next command you type works on those records instead of the whole file**. That one sentence is most of this page.

SD folds case, so a command may be typed in either case. Commands and keywords are shown here in lower case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

Every listing on this page was produced by running it, on SD Core for Windows W1.0-0, in an administrator account. The file being queried is `voc`, because it is the one file every account has.

The one thing to understand first

```
select voc with dispatch = "OS"
```

```
2 record(s) selected to list 0
```

Now the file has 418 records and the list has 2. **The next command sees 2:**

```
count voc
```

```
2 record(s) counted
```

That is the point of select lists and it is also the trap. A list you built and forgot about silently narrows the next thing you do, and nothing about the answer says so — `2 record(s) counted` is not obviously wrong. **A surprisingly small answer is very often a leftover list.**

The list is consumed by the command that uses it. Run `count voc` again and you get 418, because the list is gone.

THE PROMPT TELLS YOU

While a list is active the command prompt gains a second colon:

```
:select voc with dispatch = "OS"
2 record(s) selected to list 0
::
```

`::` **means a list is waiting.** `:` means there is not one. This is the quickest way to answer *why did that only find two records* before you go looking for a bug.

`clear.select` throws lists away without using them:

<code>clear.select {n}</code>	discard list <i>n</i>
<code>clear.select all</code>	discard all of them

```
clear.select all
```

```
Cleared all numbered select lists
```

`clearselect` is the same verb, spelled without the dot.

Building a list

<code>select</code> <i>file {selection}</i>	the records that match
<code>sselect</code> <i>file {selection}</i>	the same, sorted
<code>show</code> <i>file {selection}</i>	choose records interactively
<code>qselect</code> {dict} <i>file {saving n}</i>	a list of field values rather than record ids
<code>nselect</code> <i>file</i>	keep only the ids of the active list that are not in <i>file</i>
<code>form.list</code> {dict} <i>file record</i>	take the list from a record's contents

`select` and `sselect` take the whole selection language of the query processor — `with`, the operators, `by` — so anything you can list you can select. See SD TCL - The Query Processor.

`qselect` **reads a field out of each record and makes a list of those values.** It is how you follow a reference from one file into another: select the orders, `qselect` the customer number, and the list you are left with is customers.

`qselect` **does not tell you which list it filled.** Its completion message ends after the words *select list* with no number: `14 record(s) selected to select list`. **The list is there** — it is list 0 unless you gave `to n`. `select` and `nselect` both print the number correctly, so do not read this as the list being missing.

Numbered lists

There are **eleven**, numbered **0 to 10**. List 0 is the default and the one every verb uses when you do not say otherwise. `to n` puts the result somewhere else, which is how you keep two lists at once:

```
select voc with dispatch = "OS" to 3
select voc with dispatch = "IN" to 4
```

```
2 record(s) selected to list 3
45 record(s) selected to list 4
```

A list in 1 to 10 does not affect the next command. Only list 0 does. That is what makes the numbered lists a safe scratch area, and it is why the prompt above stayed `:` rather than becoming `::`.

Saving a list by name

Numbered lists last as long as your session. A **saved** list is a record in the account's `$savedlists` file and outlives it.

<code>save.list</code> { <i>name</i> } { from <i>n</i> }	save list <i>n</i> as <i>name</i>
<code>get.list</code> { <i>name</i> } { to <i>n</i> }	load it back into list <i>n</i>
<code>delete.list</code> <i>name</i>	delete the saved list
<code>copy.list</code> <i>name</i> { <code>,</code> <i>new</i> } { from <i>file</i> } { to <i>file</i> }	copy one, or <code>*</code> for all

Both default to list 0, and `save.list` defaults the name to the list number.

```
select voc with dispatch = "OS"
save.list zzos
```

```
2 record(s) selected to list 0
2 records saved to select list 'zzos'
```

```
get.list zzos
delete.list zzos
```

```
2 record(s) selected to select list 0
Deleted saved select list 'zzos'
```

`copy.list` **moves a saved list between accounts**, because `from` and `to` name the `$savedlists` file to read and write. Within one account it renames or duplicates.

Combining two lists

There are two sets of verbs and they take different things. this is the easiest mistake on the page.

TAKES

<code>list.union</code> · <code>list.inter</code> · <code>list.diff</code>	saved list names	reads and writes <code>\$savedlists</code>
--	-------------------------	--

<code>merge.list</code>	list numbers	works on the numbered lists in your session
-------------------------	---------------------	---

Giving a number to `list.union` does not do the numbered thing — it looks for a saved list with that name and fails:

```
list.union 3 4 5
```

```
First source list does not exist
```

BY NAME

```
list.union zzos zzin zzboth
list.inter zzos zzin zzover
```

```
47 records saved to select list 'zzboth'
0 records saved to select list 'zzover'
```

Both write a **saved** list, not a numbered one, so `get.list` is how you pick the result up. The two answers above check each other: the sources hold 2 and 45 records with nothing in common, so the union is 47 and the intersection is 0.

BY NUMBER

```
merge.list 3 union 4 to 5
merge.list 3 intersect 4 to 6
```

```
47 record(s) selected to select list 5
0 record(s) selected to select list 6
```

The operator words are `union`, `intersect` (or `intersection`) and `diff` (or `difference`). `to n` names the target, defaulting to list 0, and `count.sup` suppresses the count line.

`list.diff` and `merge.list ... diff ...` give the records in the first list that are not in the second, so **the order of the two arguments matters** where it does not for union and intersection.

Who has these verbs

All of them are in a standard account. Building, saving and combining lists is reading, and nothing on this page changes a record.

See also

SD TCL - The Query Processor · SD TCL - Files and Records · SD Basic - Select Lists.